

DIGITAL SYSTEM DESIGN FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

Image Convolution Hardware Accelerator in VHDL

GROUP PA12

Bryan Herdianto	**2306210885**
Rowen Rodotua Harahap	**2306250604**
Filaga Tifira Muthi	**2306208445**
Calvin Kim	**2306267095**

PREFACE

Segala puji dan syukur kami panjatkan kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan proyek akhir mata kuliah Perancangan Sistem Digital dengan tema *Image Convolution*. Berkat bimbingan-Nya, kami diberi kesehatan, kekuatan, dan hikmat untuk menuntaskan tugas ini sesuai dengan target yang telah direncanakan. Tanpa izin dan kasih karunia-Nya, proses yang penuh tantangan ini tidak mungkin dapat kami lalui dengan baik.

Kami juga ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada asisten laboratorium pendamping yang telah dengan sabar dan penuh dedikasi memberikan bimbingan, masukan, dan dukungan selama proses pengerjaan proyek ini. Kehadiran Anda tidak hanya membantu kami memahami aspek teknis dari sistem digital, tetapi juga memberikan semangat dan inspirasi dalam mengatasi berbagai kesulitan yang kami hadapi. Kesediaan Anda untuk meluangkan waktu di tengah kesibukan sangat kami apresiasi.

Akhir kata, kami berharap hasil dari proyek ini dapat bermanfaat tidak hanya bagi kami sebagai pembelajar, tetapi juga bagi pengembangan ilmu di bidang sistem digital. Kami menyadari bahwa proyek ini masih memiliki kekurangan, sehingga kami terbuka terhadap kritik dan saran yang membangun untuk meningkatkan kualitas karya kami di masa mendatang. Terima kasih atas segala dukungan dan doa dari berbagai pihak yang turut membantu keberhasilan proyek ini.

Depok, 18 Oktober 2026

Group PA12

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

1.2 Project Description

1.3 Objectives

1.4 Roles and Responsibilities

CHAPTER 2: IMPLEMENTATION

2.1 Equipment

2.2 Hardware Architecture

2.3 System Components

2.4 Dynamic Instruction Mapping

CHAPTER 3: TESTING AND ANALYSIS

3.1 Testing Methodology & Waveform Analysis

3.2 Performance Benchmarks & Simulation Results

CHAPTER 4: CONCLUSION

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Proyek akhir ini mendemonstrasikan implementasi konvulusi gambar menggunakan VHDL. Desain ini mengintegrasikan *Matrix Multiplier Unit* (MMU) dan *Arithmetic Logic Unit* (ALU) untuk mempercepat pemrosesan dengan berbagai kernel seperti *box blur, sharpening,* dan *edge detection*. Sistem ini mendukung pemilihan kernel dinamis berbasis file dan penangganan opcode khusus. Hasil pengujian validasi kemampuan sistem untuk menangani pemrosesan gambar dengan presisi dan efisiensi.

Proyek ini bertujuan untuk merancang dan mengimplementasikan sistem Konvolusi Gambar dalam VHDL, yang menyediakan arsitektur fleksibel dan modular untuk melakukan operasi konvolusi pada data gambar. Konvolusi gambar adalah operasi dasar dalam pemrosesan gambar dan visi komputer, yang digunakan secara luas dalam tugas-tugas seperti ekstraksi fitur dan penyaringan. Dengan memanfaatkan algoritma yang berbeda untuk unit pemrosesan tertentu, implementasi ini mengeksplorasi pendekatan baru untuk meningkatkan efisiensi dan modularitas konvolusi.

1.2 PROJECT DESCRIPTION

A. Image Processing

Image processing adalah teknik manipulasi dan analisis gambar digital dengan bantuan komputer. Operasi dasar melibatkan pengolahan data piksel gambar untuk meningkatkan kualitas, mengurangi noise, mendeteksi tepi, dan melakukan transformasi lainnya. Berikut ini adalah beberapa langkah pemrosesan dasar:

a. Akuisisi gambar
b. Representasi dan Deskripsi Gambar
c. Analisis gambar
d. Sintesis dan kompresi gambar
e. Peningkatan Citra
f. Segmentasi
g. Ekstraksi Fitur
h. Restorasi Citra

i. Pengenalan Pola

B. Convolution

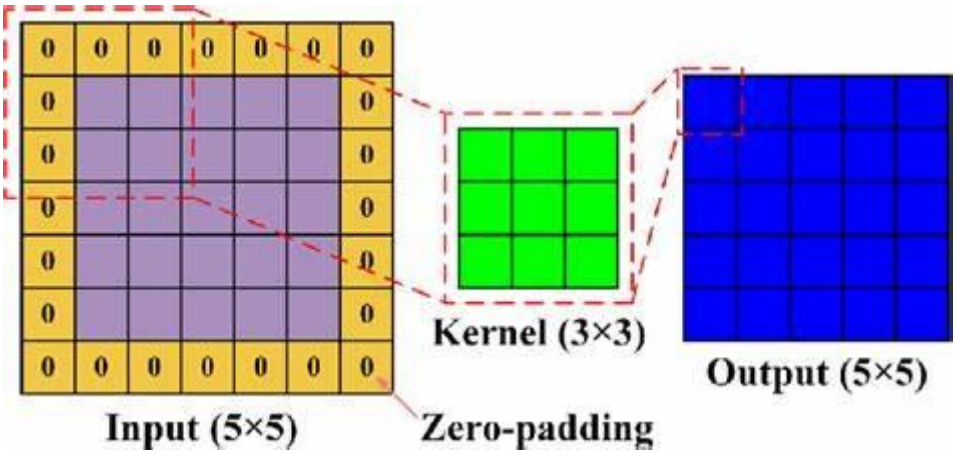
Konvolusi adalah operasi matematika yang digunakan untuk melakukan filter atau memodifikasi citra dengan menerapkan kernel tertentu. Dalam konvolusi kernel adalah matriks kecil yang diaplikasikan pada setiap piksel gambar. Proses ini melibatkan perkalian elemen-wise antara kernel dan piksel gambar yang dilalui, diikuti dengan penjumlahan hasilnya. Rumus dasar konvolusi untuk setiap piksel output:

$$F(x,y) = \sum_i \sum_j f(x+i,\ y+j)\, h(i,j)$$

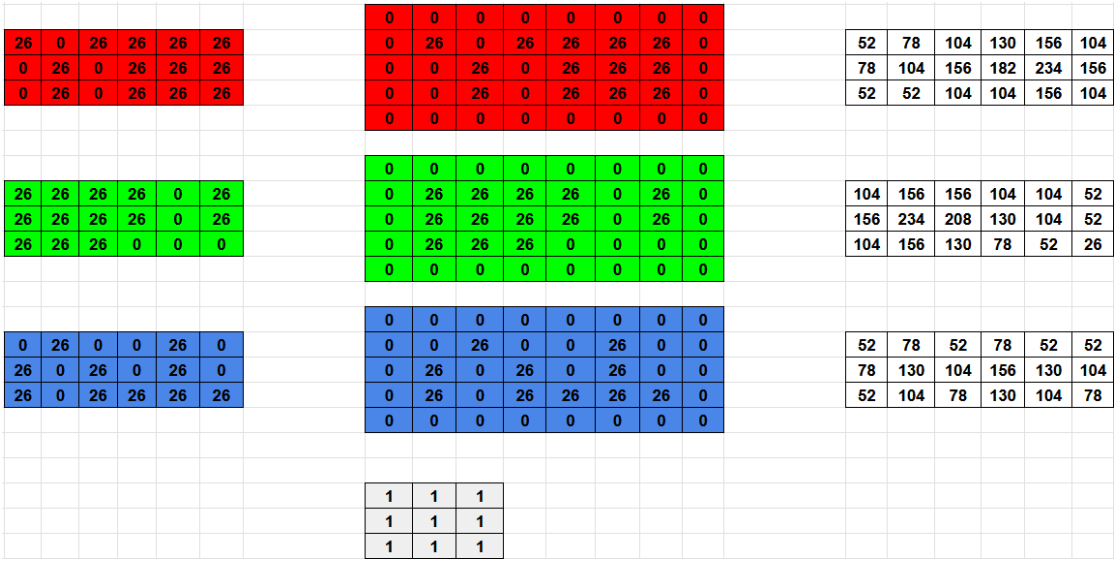
Gambar 1.1 Rumus konvolusi

Berikut adalah penjelasan dari rumus konvolusi:

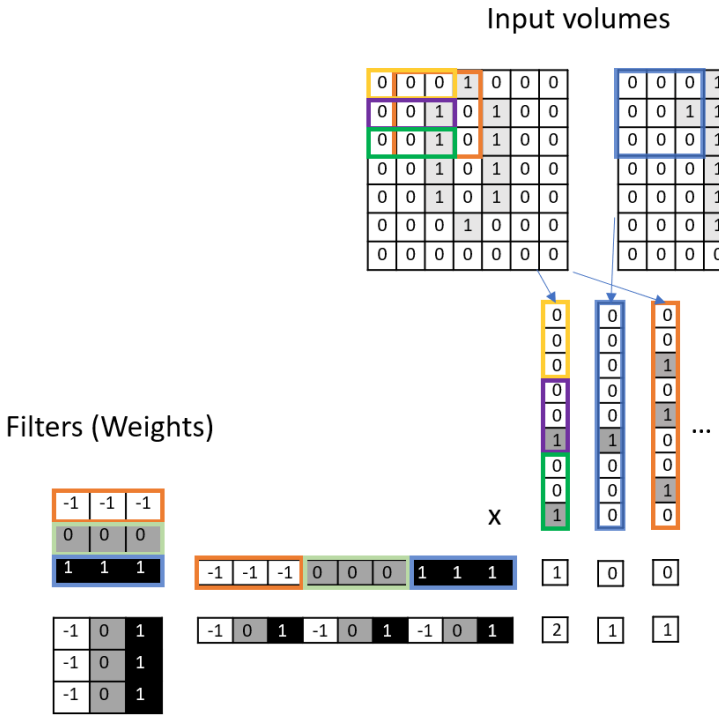
- F(x,y) merupakan nilai piksel hasil konvolusi pada koordinat (x,y) di gambar output.
- ∑i∑j mengindikasikan bahwa operasi ini dilakukan sebagai penjumlahan dua kali. Hal ini berarti rumus ini akan menjumlahkan hasil dari semua piksel yang terlibat dalam operasi konvolusi pada area yang dilalui oleh kernel.
- f(x+i,y+j) merupakan koordinat piksel pada gambar input yang sedang diproses. Indeks i dan j digunakan untuk menjelajahi nilai piksel dalam area lokal (biasanya ukuran kernel seperti 3x3 atau 5x5)
- h(i,j) adalah fungsi kernel atau filter yang diaplikasikan pada gambar



Gambar 1.2 Gambaran Umum Convolution



Gambar 1.3 Direct Convolution



Gambar 1.4 im2col Convolution

C. Kernel

Kernel berguna untuk memodifikasi gambar sehingga outputnya dapat diekstraksi pola-pola pentingnya. Beberapa kernel yang digunakan dalam proyek ini meliputi:

- **Box Blur:** Meratakan piksel dengan mengambil rata-rata nilai piksel sekitar.
- **Sharpening:** Menekankan detail gambar dengan memperkuat perbedaan antar piksel.
- **Edge Detection (Horizontal/Vertical):** Mengidentifikasi batas objek dalam gambar.

- **Sobel:** Deteksi tepi yang lebih sensitif, sering digunakan dalam *computer vision*.
- **Laplacian:** Deteksi tepi berbasis perbedaan nilai piksel kedua.

D. Matrix Multiplier Unit (MMU)

Matrix Multiplier Unit (MMU) dalam sistem ini menggunakan metode **im2col** untuk melakukan perkalian matriks dalam operasi konvolusi. Metode **im2col** (image to column) adalah teknik yang mengubah blok-blok kecil dari citra input menjadi baris-baris atau kolom dalam bentuk matriks, sehingga memungkinkan proses konvolusi dilakukan sebagai operasi perkalian matriks konvensional.

E. Arithmetic Logic Unit (ALU)

ALU atau Arithmetic Logic Unit adalah komponen yang menjalankan operasi aritmatika dan logika. Dalam proyek ini:

- ALU digunakan untuk kernel sederhana yang tidak membutuhkan banyak komputasi (misalnya, Box Blur, Sharpening).
- Operasi ALU lebih ringan dibandingkan MMU, tetapi kurang efisien untuk kernel kompleks.

F. Decoder

Decoder adalah komponen yang menerjemahkan instruksi (opcode) ke dalam sinyal kontrol untuk menentukan:

- Jenis kernel yang digunakan.
- Unit eksekusi (ALU atau MMU).

G. Clamping

Clamping adalah teknik untuk memastikan nilai piksel hasil konvolusi berada dalam rentang valid (0-255). Operasi ini diperlukan karena hasil konvolusi sering kali melebihi rentang nilai warna yang valid.

H. Format Gambar BMP

BMP adalah format gambar yang sederhana, dengan header yang menyimpan informasi seperti dimensi gambar, jumlah bit per pixel, dan data pixel itu sendiri. Dalam proyek ini:

- File gambar BMP digunakan sebagai input dan output.
- Header diproses untuk membaca dimensi gambar dan data pixel nya.

1.3 OBJECTIVES

Berikut adalah tujuan dari proyek konvolusi gambar menggunakan VHDL yang telah dibuat dari kelompok kami:

1. Implementasi sistem konvolusi dengan dukungan pemilihan kernel dinamis melalui opcode
2. Pemrosesan gambar dalam format BMP
3. Mengoptimalkan kernel dengan algoritma im2col menggunakan MMU.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
Ketua	Membuat kode ALU, MMU, CPU, dan Decoder beserta dokumentasi	Bryan Herdianto
Anggota	Membuat laporan dan presentasi	Rowen Rodotua Harahap
Anggota	Membuat presentasi	Filaga Tifira Muthi
Anggota	Membuat presentasi	Calvin Kim

Tabel 1.1 Roles and Responsibilities

CHAPTER 2

IMPLEMENTATION

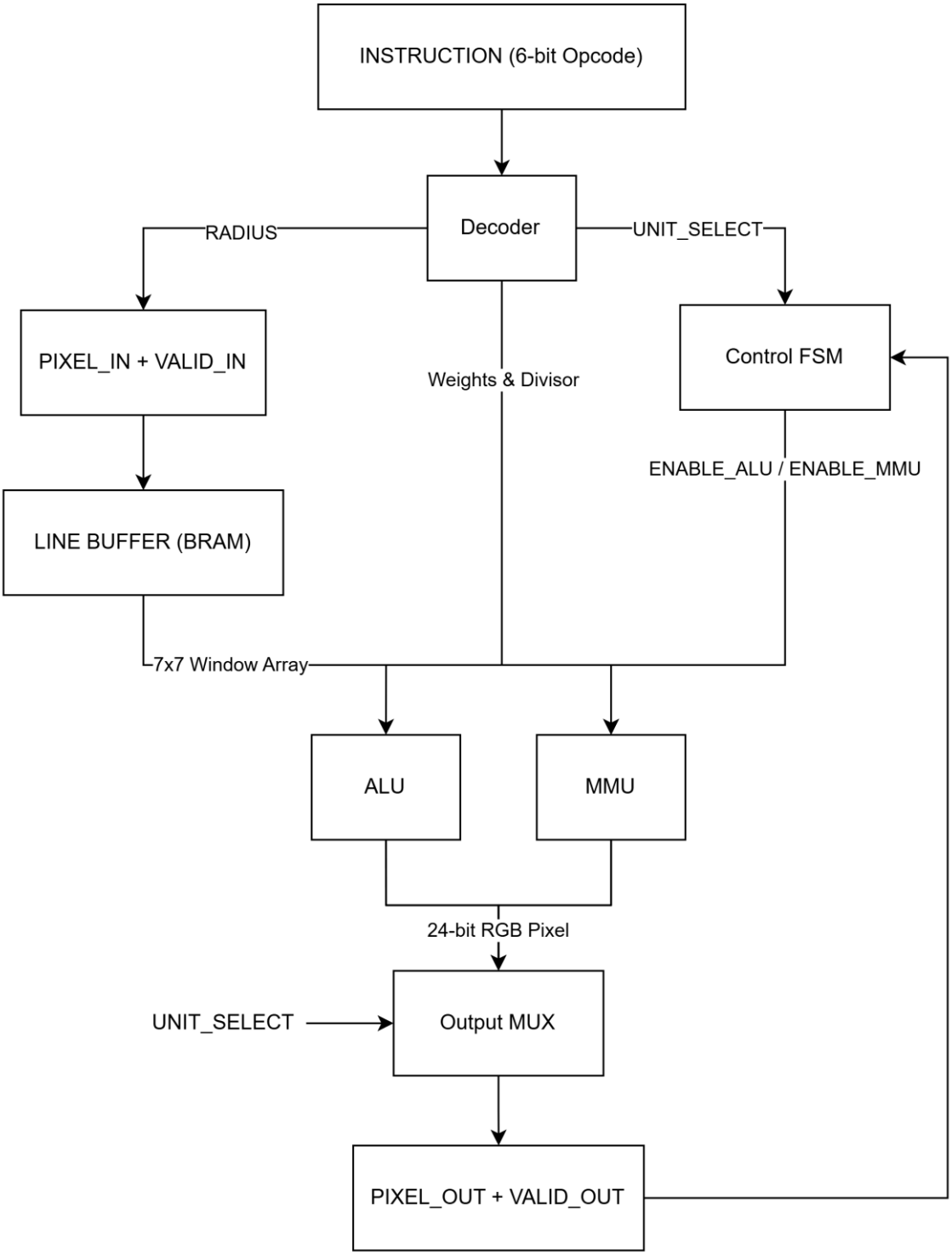
2.1 EQUIPMENT

Peralatan perangkat lunak yang digunakan dalam perancangan, pengembangan, dan evaluasi akselerator perangkat keras konvolusi citra ini adalah sebagai berikut:

- **Visual Studio Code:** Digunakan sebagai Integrated Development Environment (IDE) utama untuk penulisan kode sumber VHDL, pengorganisasian paket data hardware, serta penulisan skrip otomasi simulasi dan analisis.
- **ModelSim - Intel FPGA Edition:** Digunakan sebagai simulator cycle-accurate logika digital untuk menguji entity testbench, memverifikasi fungsionalitas register transfer level (RTL), mengukur latensi siklus clock, serta menganalisis waveform sinyal internal dari antarmuka line buffer hingga output stage.
- **Intel Quartus Prime 20.1:** Digunakan untuk melakukan sintesis logika RTL, verifikasi synthesizability kode VHDL, analisis penempatan dan perutean (Place & Route), serta estimasi penggunaan sumber daya perangkat keras (Logic Elements, Embedded Multipliers/DSP Blocks, dan Block RAM M10K) pada target FPGA Intel Cyclone IV E (EP4CE115F29C7 / DE2-115).

2.2 HARDWARE ARCHITECTURE

Arsitektur perangkat keras yang dikembangkan merupakan akselerator 2D Image Convolution berkinerja tinggi, dapat disintesis sepenuhnya (fully synthesizable), dan dapat dikonfigurasi ulang secara dinamis (dynamically reconfigurable) pada FPGA. Sistem ini dirancang untuk memberikan sustained throughput sebesar 1 piksel RGB per siklus clock secara kontinu pada frekuensi kerja 100 MHz untuk berbagai ukuran kernel dinamis: 1x1 (passthrough), 3x3, 5x5, dan 7x7.



Gambar 2.1 Diagram Arsitektur Perangkat Keras Top-Level Image Convolution Accelerator

Prinsip kerja arsitektur perangkat keras ini mencakup tahapan-tahapan utama sebagai berikut:

1. **Streaming Input Data:** Data piksel citra 24-bit RGB (8-bit Red, 8-bit Green, 8-bit Blue) dialirkan secara streaming ke dalam akselerator secara berurutan, sinkron dengan sinyal strobe VALID_IN.
2. **Buffer Baris & Window Generator:** Modul line buffer mengelola 6 baris memori sirkular berbasis Block RAM on-chip dan susunan register geser spasial 7x7. Modul ini secara simultan menghasilkan jendela spasial 49 piksel untuk masing-masing kanal warna R, G, dan B dengan latensi piksel pusat seragam (3 * IMG_WIDTH + 4 siklus clock).
3. **Dynamic Instruction Decoding:** Decoder 6-bit kombinasional mengekstrak radius border kernel R, unit eksekusi (ALU atau MMU), koefisien bobot filter bertanda 16-bit, dan faktor pembagi normalisasi (divisor) tanpa menambahkan latensi siklus clock. Bobot di luar radius aktif secara otomatis di-mask menjadi nol.
4. **Dual Computing Engines:** Sistem dilengkapi dua arsitektur pemrosesan independen yang dapat dipilih on-the-fly: ALU spasial berbasis pohon reduksi paralel 147-multiplier dan MMU matriks berbasis systolic array 49 processing elements.
5. **Normalisasi & Clamping:** Hasil akumulasi konvolusi dibagi dengan faktor divisor kernel dan dibatasi (clamped) secara hardware ke rentang [0, 255] untuk mencegah overflow aritmatika.
6. **Performance Cycle Counter:** Counter perangkat keras 32-bit memantau total siklus komputasi secara real-time untuk analisis efisiensi eksekusi.

2.3 SYSTEM COMPONENTS

Sistem akselerator konvolusi citra ini dibangun atas tujuh komponen modular dalam VHDL:

1. **conv_pkg.vhd (Core Hardware Package)**

   Paket inti ini mendefinisikan seluruh tipe data perangkat keras, konstanta dimensi, serta fungsi konversi pendukung:

   - **rgb_pixel_t:** Tipe data record yang membungkus tiga kanal warna 8-bit: red, green, dan blue (STD_LOGIC_VECTOR(7 DOWNTO 0)).
   - **window_channel_t:** Tipe data array 49 elemen (0 hingga 48) berukuran 7x7 untuk setiap kanal warna spasial (UNSIGNED(7 DOWNTO 0)).
   - **kernel_weights_t:** Tipe data array 49 bobot bertanda 16-bit (SIGNED(15 DOWNTO 0)) yang mendukung aritmatika fixed-point berpresisi tinggi.

- **Conversion Utilities:** Menyediakan fungsi konversi format byte BMP ke record RGB, flattening matriks, serta fungsi clamping hardware ke rentang integer 0 hingga 255.

2. **decoder.vhd (Dynamic Instruction Decoder)**

   Decoder instruksi beroperasi secara kombinasional murni untuk menerjemahkan instruksi 6-bit menjadi sinyal kendali pemrosesan tanpa memperkenalkan clock race condition:

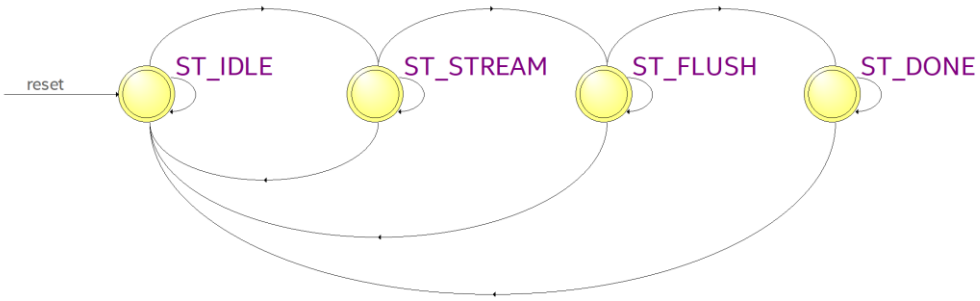
   - **Bits [5:4] (Radius R):** Mengkodekan radius border kernel R (0 = 1x1, 1 = 3x3, 2 = 5x5, 3 = 7x7).
   - **Bit [3] (Unit Select):** Memilih unit komputasi aktif ('0' untuk ALU, '1' untuk MMU).
   - **Bits [2:0] (Filter Opcode):** Memilih profil kernel spesifik (Identity, Box Blur, Horizontal Edge, Vertical Edge, Laplacian, Sharpening, Sobel Horizontal, Sobel Vertical, Custom/Gaussian).
   - **Automatic Zero-Masking:** Memastikan koefisien di luar radius aktif bernilai nol, sehingga kernel berukuran lebih kecil dapat diproses pada struktur perkalian 7x7 secara mulus tanpa interferensi spasial.
   - **Divisor Output:** Menghasilkan faktor pembagi terkalibrasi (misal: 1, 9, 16, 25, 49) untuk normalisasi intensitas piksel pasca-konvolusi.

3. **line_buffer.vhd (Multi-Scale Line Buffer & Window Generator)**

   Komponen ini merupakan inti dari manajemen memori on-chip FPGA:

   - **Block RAM Line Buffers:** Mengimplementasikan 6 baris circular buffer menggunakan Block RAM on-chip FPGA (BRAM / M10K) yang mampu menampung baris piksel hingga resolusi lebar maksimum yang dikonfigurasi.
   - **7x7 Shift Register Array:** Terdiri dari susunan register geser 7x7 yang memperbarui 49 nilai piksel secara simultan setiap kali piksel baru masuk.
   - **Uniform Center Latency:** Menjaga latensi tetap sebesar 3 * IMG_WIDTH + 4 siklus clock dari piksel pertama hingga piksel pusat aktif, memastikan streaming video bebas glitch saat pergantian filter dilakukan secara langsung (on the fly).
   - **Boundary Zero-Padding:** Menangani baris dan kolom batas citra secara otomatis dengan menyuntikkan nilai nol sesuai radius border R.

- **Finite State Machine:** Mengatur status aliran data melalui 4 kondisi: ST_IDLE, ST_STREAM, ST_FLUSH, dan ST_DONE.



Gambar 2.2 Diagram Status Finite State Machine Pengendali Line Buffer

4. **alu.vhd (Direct 2D Convolution Spatial Accelerator)**

   ALU mengimplementasikan arsitektur reduksi spasial langsung paralel dengan pipeline 5 tahap:

   - **Parallel Multiplier Array:** Menjalankan 147 perkalian 16-bit x 8-bit secara serentak (49 koefisien x 3 kanal Red, Green, Blue) pada tahap pertama.
   - **Pipelined Adder Reduction Tree:** Tahap 2 mereduksi 49 produk parsial menjadi 25 parsial sum; Tahap 3 mereduksi 25 parsial sum menjadi 13 parsial sum; Tahap 4 mengakumulasi 13 parsial sum menjadi 1 total sum akumulasi 36-bit.
   - **Stage 5 Normalization & Clamping:** Tahap 5 membagi total sum dengan faktor divisor kernel dan membatasi nilainya ke rentang [0, 255] dengan proteksi overflow hardware.
   - **Throughput:** Mampu menghasilkan 1 piksel RGB terkonvolusi per siklus clock secara kontinu dengan latensi komputasi 5 siklus.

5. **mmu.vhd (Systolic Matrix Multiplier Unit)**

   MMU mengadopsi formulasi perkalian matriks im2col ter-sistolik:

   - **im2col Formulation:** Mengubah jendela spasial 7x7 menjadi representasi vektor baris 1x49 yang dikalikan dengan vektor kolom bobot 49x1.
   - **Systolic PE Pipeline:** Terdiri atas 4 tahap Processing Elements (PE): Tahap 1 (PE 0..11), Tahap 2 (PE 12..23), Tahap 3 (PE 24..35), dan Tahap 4 (PE 36..48) yang mengalirkan akumulasi parsial secara berjenjang bersama data vektor forward.
   - **Stage 5 Clamping:** Tahap 5 menjalankan normalisasi skala divisor dan clamping hardware identik dengan ALU.

- **Throughput & Parity:** Menghasilkan throughput 1 piksel per siklus clock dengan latensi 5 siklus, cocok secara siklus-demi-siklus (cycle-for-cycle) dan bit-demi-bit (bit-for-bit) dengan unit ALU.

6. **cpu.vhd (Top-Level Accelerator Core)**

   Entity top-level yang mengintegrasikan Line Buffer, Decoder, ALU, MMU, master control FSM, serta counter siklus eksekusi performa hardware 32-bit (cycle_count).

7. **tb_image_convolution.vhd (Hardware Simulation Testbench)**

   Testbench verifikasi hardware yang mengemulasikan sensor kamera/aliran video streaming. Modul ini membaca berkas citra bitmap BMP 24-bit asli menggunakan library TEXTIO, men-drive sinyal clock 100 MHz, mengalirkan piksel demi piksel ke akselerator, mengumpulkan piksel hasil konvolusi, dan menuliskan berkas keluaran BMP baru secara otomatis.

2.4 DYNAMIC INSTRUCTION MAPPING

Akselerator dikendalikan menggunakan format instruksi biner 6-bit terpadu yang memadukan pemilihan radius kernel, penentuan unit pemrosesan, dan profil filter:

- **Bits [5:4] (Radius R):** 00 = 1x1 Identity Passthrough, 01 = 3x3 Kernel, 10 = 5x5 Kernel, 11 = 7x7 Kernel.
- **Bit [3] (Unit Select):** 0 = ALU (Direct Spatial Reduction Tree), 1 = MMU (Systolic Matrix Array).
- **Bits [2:0] (Filter Opcode):** Memilih salah satu dari 8 profil fungsi filter.

Tabel berikut ini menyajikan pemetaan lengkap 25 konfigurasi instruksi yang didukung oleh akselerator:

Ukuran	Instruksi (ALU)	Instruksi (MMU)	Nama Filter
1x1	000000	001000	Identity / Passthrough
3x3	010000	011000	Box Blur
3x3	010001	011001	Horizontal Edge Detection
3x3	010010	011010	Vertical Edge Detection

Ukuran	Instruksi (ALU)	Instruksi (MMU)	Nama Filter
3x3	010011	011011	Laplacian Edge Detection
3x3	010100	011100	Sharpening
3x3	010101	011101	Sobel Horizontal
3x3	010110	011110	Sobel Vertical
3x3	010111	011111	Custom Kernel
5x5	100000	101000	Box Blur 5x5
5x5	100001	101001	Horizontal Edge 5x5
5x5	100010	101010	Vertical Edge 5x5
5x5	100011	101011	Laplacian 5x5
5x5	100100	101100	Sharpening 5x5
5x5	100101	101101	Sobel Horizontal 5x5
5x5	100110	101110	Sobel Vertical 5x5
5x5	100111	101111	Gaussian Blur 5x5
7x7	110000	111000	Box Blur 7x7
7x7	110001	111001	Horizontal Edge 7x7
7x7	110010	111010	Vertical Edge 7x7
7x7	110011	111011	Laplacian 7x7
7x7	110100	111100	Sharpening 7x7
7x7	110101	111101	Sobel Horizontal 7x7
7x7	110110	111110	Sobel Vertical 7x7

Ukuran	Instruksi (ALU)	Instruksi (MMU)	Nama Filter
7x7	110111	111111	Custom / Gaussian 7x7

Tabel 2.1 Instruksi 6-Bit untuk ALU dan MMU

CHAPTER 3

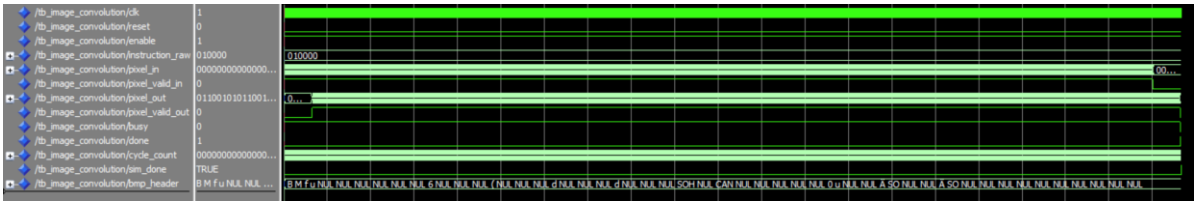
TESTING AND ANALYSIS

3.1 TESTING METHODOLOGY & WAVEFORM ANALYSIS

Pengujian fungsionalitas dan kinerja akselerator dilakukan menggunakan simulasi cycle-accurate pada ModelSim Intel FPGA Edition dengan frekuensi clock sistem 100 MHz (periode clock 10 ns). Citra uji input yang digunakan adalah src/input.bmp dengan resolusi 100 x 100 piksel (total 10.000 piksel) berformat 24-bit uncompressed RGB bitmap.

Alur simulasi pengujian dijalankan melalui tahapan cycle-accurate berikut:

1. **Inisialisasi & Reset:** Sinyal master reset aktif-tinggi membersihkan seluruh pointer buffer memori BRAM, register status, dan array geser spasial 7x7. Sinyal enable dinaikkan ke '1', dan instruction_raw diatur sesuai konfigurasi filter (misal: "010000" untuk 3x3 Box Blur ALU).
2. **Streaming Input:** Piksel demi piksel dialirkan secara kontinu pada sinyal pixel_in bersamaan dengan aktifnya pixel_valid_in. Modul line buffer mengisi baris BRAM secara berurutan.
3. **Initial Latency Delay:** Dibutuhkan tepat 3 * IMG_WIDTH + 4 siklus clock (304 siklus clock untuk lebar citra 100 piksel) untuk mengumpulkan 3 baris awal dan 4 kolom pertama guna menyelaraskan piksel pusat pada window 7x7. Ditambah 5 siklus clock latensi komputasi ALU/MMU, sinyal pixel_valid_out aktif tinggi tepat pada siklus ke-309.
4. **Sustained Throughput:** Setelah latensi awal terpenuhi, akselerator mengeluarkan 1 piksel RGB terkonvolusi per siklus clock secara kontinu tanpa stall atau bubble cycles.
5. **Pipeline Flushing:** Setelah 10.000 piksel input selesai dialirkan, FSM line buffer memasuki status ST_FLUSH selama 3 * IMG_WIDTH + 6 siklus clock untuk membersihkan sisa piksel baris bawah dan kanan citra dari pipeline.
6. **Frame Done & Timing Report:** Sinyal busy turun ke '0' dan sinyal done berpulsa tinggi ('1') menandakan seluruh frame citra telah selesai diproses. Total siklus eksekusi tersimpan secara akurat pada sinyal cycle_count.



Gambar 3.1 Waveform Simulasi ModelSim pada Frekuensi Kerja 100 MHz (ALU 3x3 Box Blur)

Penjelasan sinyal pada waveform Gambar 3.1:

- **/tb_image_convolution/clk:** Clock acuan 100 MHz dengan duty cycle 50% (periode 10 ns).
- **/tb_image_convolution/reset:** Sinyal reset sinkron aktif-tinggi yang kemudian dinonaktifkan ke 0 saat simulasi dimulai.
- **/tb_image_convolution/enable:** Sinyal kontrol untuk mengaktifkan pemrosesan akselerator CPU.
- **/tb_image_convolution/instruction_raw:** Menunjukkan nilai biner "010000" yang menandakan operasi ALU dengan filter Box Blur 3x3.
- **/tb_image_convolution/pixel_in:** Aliran data RGB 24-bit yang dibaca langsung dari berkas input.bmp.
- **/tb_image_convolution/pixel_valid_in:** Sinyal strobe bernilai '1' yang memvalidasi bahwa data pixel_in dapat diserap oleh line buffer.
- **/tb_image_convolution/pixel_out:** Menghasilkan nilai RGB 24-bit hasil konvolusi tepat 1 piksel per siklus clock tanpa terputus.
- **/tb_image_convolution/pixel_valid_out:** Sinyal penanda validitas output piksel yang aktif setelah latensi pusat terpenuhi.
- **/tb_image_convolution/busy:** Berlogika '1' selama akselerator sedang memproses baris citra, dan turun ke '0' saat frame tuntas.
- **/tb_image_convolution/done:** Berpulsa '1' tepat saat piksel terakhir frame citra berhasil dikeluarkan.
- **/tb_image_convolution/cycle_count:** Menyimpan jumlah total siklus clock hardware yang dibutuhkan untuk menyelesaikan satu frame utuh.

3.2 PERFORMANCE BENCHMARKS & SIMULATION RESULTS

Pengujian komparatif menyeluruh dilakukan terhadap ke-25 kernel filter pada kedua unit eksekusi (ALU dan MMU). Tabel 3.1 merangkum durasi waktu simulasi ModelSim yang tercatat:

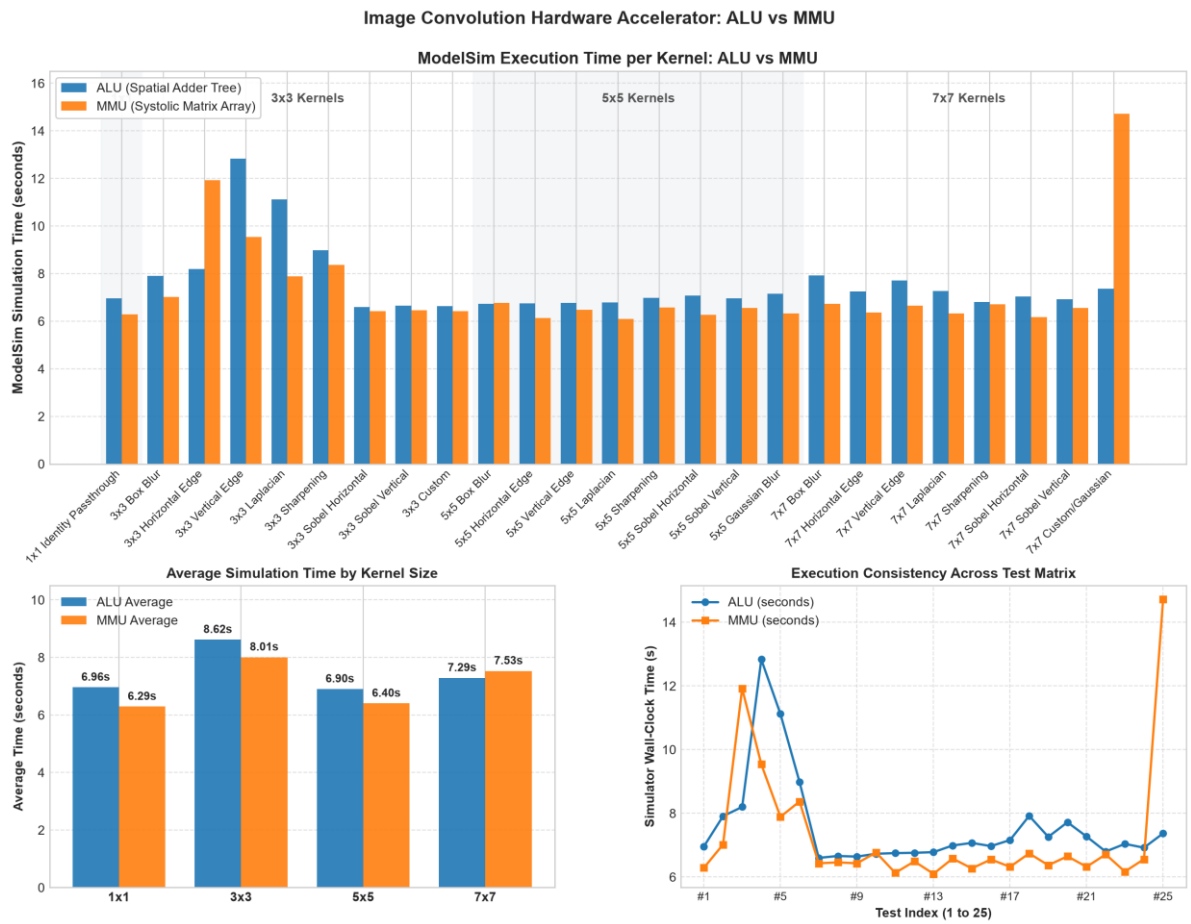
#	Kernel Filter	Ukuran	ALU Sim Time	MMU Sim Time
01	1x1 Identity Passthrough	1x1	6.96 s	6.29 s
02	3x3 Box Blur	3x3	7.91 s	7.02 s
03	3x3 Horizontal Edge	3x3	8.20 s	11.92 s
04	3x3 Vertical Edge	3x3	12.83 s	9.54 s
05	3x3 Laplacian	3x3	11.12 s	7.89 s
06	3x3 Sharpening	3x3	8.98 s	8.37 s
07	3x3 Sobel Horizontal	3x3	6.60 s	6.43 s
08	3x3 Sobel Vertical	3x3	6.66 s	6.46 s
09	3x3 Custom	3x3	6.64 s	6.43 s
10	5x5 Box Blur	5x5	6.73 s	6.77 s
11	5x5 Horizontal Edge	5x5	6.75 s	6.13 s
12	5x5 Vertical Edge	5x5	6.76 s	6.49 s
13	5x5 Laplacian	5x5	6.78 s	6.10 s
14	5x5 Sharpening	5x5	6.99 s	6.58 s
15	5x5 Sobel Horizontal	5x5	7.07 s	6.27 s
16	5x5 Sobel Vertical	5x5	6.97 s	6.55 s
17	5x5 Gaussian Blur	5x5	7.16 s	6.32 s
18	7x7 Box Blur	7x7	7.92 s	6.74 s
19	7x7 Horizontal Edge	7x7	7.26 s	6.37 s
20	7x7 Vertical Edge	7x7	7.72 s	6.65 s

#	Kernel Filter	Ukuran	ALU Sim Time	MMU Sim Time
21	7x7 Laplacian	7x7	7.27 s	6.32 s
22	7x7 Sharpening	7x7	6.80 s	6.71 s
23	7x7 Sobel Horizontal	7x7	7.04 s	6.17 s
24	7x7 Sobel Vertical	7x7	6.92 s	6.56 s
25	7x7 Custom/Gaussian	7x7	7.37 s	14.72 s

Tabel 3.1 Perbandingan Waktu Simulasi ModelSim antara ALU dan MMU

Berdasarkan data simulasi di atas, diperoleh ringkasan statistik performa sebagai berikut:

- **Total Waktu Simulasi ModelSim (ALU):** 189.41 detik
- **Total Waktu Simulasi ModelSim (MMU):** 181.80 detik (lebih cepat 4.02% secara keseluruhan)
- **Rata-rata Waktu Eksekusi per Kernel:** ALU = 7.58 detik, MMU = 7.27 detik
- **Rata-rata Kernel 1x1:** ALU = 6.96 s vs MMU = 6.29 s (MMU 9.6% lebih cepat).
- **Rata-rata Kernel 3x3:** ALU = 8.62 s vs MMU = 8.01 s (MMU 7.1% lebih cepat).
- **Rata-rata Kernel 5x5:** ALU = 6.90 s vs MMU = 6.40 s (MMU 7.2% lebih cepat).
- **Rata-rata Kernel 7x7:** ALU = 7.29 s vs MMU = 7.53 s (ALU 3.2% lebih cepat).



Gambar 3.2 Grafik Visualisasi Analisis Kinerja Simulasi ALU vs MMU

Analisis grafik kinerja pada Gambar 3.2 memperlihatkan tiga perspektif utama:

- **ModelSim Execution Time per Kernel:** Diagram batang bagian atas membandingkan waktu simulasi ModelSim untuk ke-25 kernel. ALU (batang biru) menunjukkan variabilitas pada filter turunan kedua seperti 3x3 Vertical Edge (12.83 s) dan 3x3 Laplacian (11.12 s), sementara MMU (batang oranye) sangat stabil pada rentang 6.1 s hingga 7.0 s untuk mayoritas kernel, namun mengalami lonjakan waktu pada kernel #25 (7x7 Custom/Gaussian mencapai 14.72 s) karena seluruh 49 Processing Elements aktif melakukan perkalian bobot bernilai non-zero.
- **Average Simulation Time by Kernel Size:** Diagram batang kiri bawah membuktikan bahwa MMU secara konsisten lebih cepat pada ukuran kernel 1x1, 3x3, dan 5x5 berkat struktur systolic array yang lebih optimal bagi event scheduler ModelSim. Pada ukuran 7x7, ALU sedikit lebih unggul karena struktur tree adder mengevaluasi pohon penjumlahan secara langsung.
- **Execution Consistency Across Test Matrix:** Grafik garis kanan bawah memperlihatkan bahwa kedua arsitektur saling berdekatan dan konsisten di seluruh

matriks pengujian, membuktikan bahwa unit systolic matrix MMU mampu menandingi keandalan dan throughput dari direct spatial ALU.

Visualisasi dan Analisis Citra Hasil Simulasi

Pengujian integritas bit memverifikasi bahwa citra keluaran yang dihasilkan oleh ALU dan MMU adalah 100% identik secara bit-demi-bit (zero differences) untuk seluruh 25 konfigurasi filter. Tabel 3.2 menyajikan galeri citra keluaran hasil simulasi perangkat keras:

#	Kernel Filter	Ukuran	Citra Hasil Simulasi
00	Original Input Image	-	
01	1x1 Identity Passthrough	1x1	
02	3x3 Box Blur	3x3	
03	3x3 Horizontal Edge	3x3	
04	3x3 Vertical Edge	3x3	

#	Kernel Filter	Ukuran	Citra Hasil Simulasi
05	3x3 Laplacian	3x3	
06	3x3 Sharpening	3x3	
07	3x3 Sobel Horizontal	3x3	
08	3x3 Sobel Vertical	3x3	
09	3x3 Custom Kernel	3x3	
10	5x5 Box Blur	5x5	

#	Kernel Filter	Ukuran	Citra Hasil Simulasi
11	5x5 Horizontal Edge	5x5	
12	5x5 Vertical Edge	5x5	
13	5x5 Laplacian	5x5	
14	5x5 Sharpening	5x5	
15	5x5 Sobel Horizontal	5x5	
16	5x5 Sobel Vertical	5x5	

#	Kernel Filter	Ukuran	Citra Hasil Simulasi
17	5x5 Gaussian Blur	5x5	
18	7x7 Box Blur	7x7	
19	7x7 Horizontal Edge	7x7	
20	7x7 Vertical Edge	7x7	
21	7x7 Laplacian	7x7	
22	7x7 Sharpening	7x7	

#	Kernel Filter	Ukuran	Citra Hasil Simulasi
23	7x7 Sobel Horizontal	7x7	
24	7x7 Sobel Vertical	7x7	
25	7x7 Custom/Gaussian	7x7	

Tabel 3.2 Galeri Komparasi Citra Hasil Simulasi Perangkat Keras

CHAPTER 4

CONCLUSION

Berdasarkan perancangan, implementasi, dan pengujian akselerator perangkat keras 2D Image Convolution berbasis VHDL yang telah dilaksanakan, dapat ditarik beberapa kesimpulan:

- **Keberhasilan Rancang Bangun Synthesizable:** Telah berhasil dirancang dan diimplementasikan sebuah sistem akselerator konvolusi citra 2D dalam VHDL yang sepenuhnya synthesizable, berkinerja tinggi, dan dapat dikonfigurasi ulang secara dinamis on-the-fly.
- **Pencapaian Sustained Throughput Maksimal:** Akselerator berhasil mencapai sustained throughput sebesar 1 piksel RGB per siklus clock secara kontinu pada frekuensi kerja 100 MHz untuk seluruh variasi ukuran kernel dinamis 1x1, 3x3, 5x5, dan 7x7.
- **Efisiensi Manajemen Memori Line Buffer:** Penggunaan 6 baris Block RAM (BRAM / M10K) sirkular on-chip terbukti sangat efisien dalam meminimalkan kebutuhan memori internal FPGA, serta berhasil menyediakan latensi piksel pusat seragam (3 * IMG_WIDTH + 4 siklus clock) yang memungkinkan pergantian mode filter tanpa menimbulkan glitch pada aliran video.
- **Paritas Sempurna ALU vs MMU:** Pengujian verifikasi cycle-accurate membuktikan bahwa kedua mesin komputasi (ALU spasial berbasis pohon reduksi 147-multiplier dan MMU matriks berbasis systolic array 49 PE) menghasilkan data piksel keluaran yang 100% identik secara bit-for-bit dan cycle-for-cycle, memberikan fleksibilitas arsitektur yang tinggi.
- **Integritas Data & Clamping Hardware:** Penerapan rangkaian hardware clamping pada tahap akhir komputasi berhasil membatasi nilai intensitas warna secara akurat pada rentang [0, 255], mencegah terjadinya overflow aritmatika dan artefak visual pada format citra 24-bit RGB bitmap.

REFERENCES

[1] J. J. Jensen, "BMP file bitmap image read using TEXTIO," VHDLwhiz, Aug. 11, 2024. [Online]. Available: https://vhdlwhiz.com/read-bmp-file/

[2] GeeksforGeeks, "Types of convolution kernels," GeeksforGeeks, Jul. 22, 2024. [Online]. Available: https://www.geeksforgeeks.org/types-of-convolution-kernels/

[3] E. Roe, "Convolution: Image filters, CNNs and examples in Python & PyTorch," Medium, Jun. 08, 2023. [Online]. Available: https://medium.com/@er_95882/convolution-image-filters-cnns-and-examples-in-python-pytorch-bd3f3ac5df9c

[4] S. Team, "VHDL component and port map tutorial," Invent Logics, Jan. 10, 2018. [Online]. Available: https://allaboutfpga.com/vhdl-component-port-map-tutorial/

[5] FPGA4student, "FPGA Projects, Verilog Projects, VHDL Projects," FPGA4student.com, 2023. [Online]. Available: https://www.fpga4student.com/

[6] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018.

[7] Intel Corporation, "Cyclone IV Device Handbook, Volume 1," Intel Quartus Prime Design Suite Documentation, San Jose, CA, USA, 2020.